ScopePilot

REVIEWS 1 & 2 COMPLETED

Date: Oct 15, 2026

Consolidated Technical Report: Review 1 & Review 2 (AI Architecture & Mock Engine)

Student / Developer:	Vishal	Assigned Role:	AI / RAG / Change-Analysis Agent
Review Stages Covered:	Review 1 (Architecture & Contract) • Review 2 (Mock Engine & Output)	Git Repository:	github.com/Vivek-teja/ScopePilot
Working Feature Branch:	`Vishal` (Pushed to `origin/Vishal`)	Remote Commit SHA:	`92ee490` (4 Commits ahead of `origin/main`)
Verification Status:	48/48 TESTS PASSING • 5 MOCK CASES VERIFIED		

1. Executive Summary & Problem Solved

In freelance client-contractor engagements, **scope creep** is the primary source of project delays, budget overruns, and contractual disputes. Clients frequently introduce additions informally via chat or email, and freelancers often accept them without evaluating divergence from the agreed project baseline.

> **ScopePilot AI Agent Solution:** ScopePilot introduces an automated change-analysis agent that evaluates incoming client requests against approved proposal scopes (`ProposalVersion`) and deliverables (`Deliverable`) before extra work is accepted.
>
> • **Review 1 Milestone:** Established strict architectural boundaries, inspected existing domain schemas, and authored typed Zod input/output contracts.
>
> • **Review 2 Milestone:** Built a deterministic rule-based mock engine proving reproducible contract behavior across 5 required canonical case outcomes before introducing probabilistic LLMs.

2. Review 1 Summary: Architecture, Schemas & Contracts

The AI subsystem is strictly partitioned inside `backend/src/ai/` to avoid coupling with business controllers, financial transactions, or route handlers:

Domain Entity	Code Location	Role & Attributes Leveraged by AI Agent
ProposalVersion	`src/modules/proposal/proposal.model.js`	Immutable scope snapshot: `versionNumber`, `title`, `scopeSummary`, `scopeItems: [{ title, description, estimatedHours, price }]`, `status: 'approved'`, `contentHash`.
Deliverable	`src/modules/deliverable/deliverable.model.js`	Auditable baseline units: `scopeReference` (e.g. `DEL-CAT-01`), `title`, `description`, `acceptanceCriteria: [string]`.
ChangeRequest	`src/modules/changeRequest/changeRequest.model.js`	Target persistence container with AI extensions: `aiAnalysis: { summary, confidenceScore, rawOutput, analyzedAt }`, `scopeComparison: { outOfScopeItems, inScopeItems, similarityScore }`.
Decision & Project	`src/modules/project/project.model.js`	`currentApprovedProposal`, `freelancer`, `client`. Final human sign-off via `decisionService`.

Formal Zod Input & Output Contracts (`aiAnalysis.contract.js`)

- **Input Contract (`ChangeAnalysisInputSchema`):** Requires `projectId`, `approvedProposalVersion` (version number, summary, scope items), array of `deliverables` (audit `scopeReference`, criteria), and `newClientRequest` (title, description). Rejects malformed requests upfront.
- **Output Contract (`ChangeAnalysisOutputSchema`):** Enforces structured evaluation fields: `summary`, `reviewStatus` enum, `confidenceScore` (0.0–1.0), `inScopeItems`, `outOfScopeItems`, `uncertainItems`, `affectedDeliverables`, `evidence`, `recommendations`, and mandatory `disclaimer`.

> **Review 1 Human Authority Boundary:** The AI provides an advisory draft only. Final decisions to approve, reject, or renegotiate changes remain exclusively with authenticated humans. Financial adjustments and schedule dates are computed deterministically, never hallucinated by AI.

3. Review 2 Summary: Deterministic Mock Engine & 5 Canonical Cases

Review 2 proves the contract with predictable, reproducible behavior across the 5 required real-world categories:

Case ID	Category	Incoming Request Description	Review Status	Confidence & Rationale
Case 1	**Clearly In Scope**	Checkout phone number validation & international E.164 formatting	`in_scope`	**95%** • Directly refines `DEL-CHK-01` checkout flow without unbudgeted scope additions.
Case 2	**Clearly Out of Scope**	Cryptocurrency Wallet Integration & Web3 Token Settlement	`out_of_scope`	**92%** • Introduces Web3 smart contracts and blockchain wallets not scoped in proposal.
Case 3	**Partial Overlap**	Catalog CSV Bulk Import and Automated Supplier Inventory Sync	`partially_out_of_scope`	**88%** • Touches `DEL-CAT-01` (Catalog UI) but adds bulk CSV ingestion & background sync jobs.
Case 4	**Ambiguous Request**	"Make store experience significantly faster and enhance overall UI"	`needs_clarification`	**60%** • Contains subjective words (faster, enhance); asks client for Core Web Vitals & Figma specs.
Case 5	**Insufficient Evidence**	Sync orders to internal enterprise ERP via custom warehouse connector	`uncertain`	**45%** • Baseline contains zero documentation on on-premise ERP/SAP; flags missing evidence honestly.

4. CLI Demonstration Output (Summary Table: `npm run mock:cases`)

The standalone CLI test runner (`backend/src/ai/scripts/runMockCases.js`) verifies contract compliance and bit-for-bit determinism across consecutive executions:

```
================================================================================
 SUMMARY TABLE: 5 CANONICAL CASE EVALUATION
================================================================================
┌─────────┬──────────┬─────────────────────────┬───────────────────────────────────────────┬──────────────────────────┬────
────────┬─────────┬────────────┬───────────┬───────────────┐
│ (index) │ caseId   │ category                │ requestTitle                              │ reviewStatus             │
confidence │ inScope │ outOfScope │ uncertain │ deterministic │
├─────────┼──────────┼─────────────────────────┼───────────────────────────────────────────┼──────────────────────────┼────
────────┼─────────┼────────────┼───────────┼───────────────┤
│ 0       │ 'case-1' │ 'Clearly In Scope'      │ 'Checkout phone number validation...'     │ 'in_scope'               │
'95%'      │ 1       │ 0          │ 0         │ 'YES'         │
│ 1       │ 'case-2' │ 'Clearly Out of Scope'  │ 'Cryptocurrency Wallet Integration & ...' │ 'out_of_scope'           │
'92%'      │ 0       │ 7          │ 0         │ 'YES'         │
│ 2       │ 'case-3' │ 'Partial Overlap'       │ 'Product Catalog CSV Bulk Import and ...' │ 'partially_out_of_scope' │
'88%'      │ 1       │ 4          │ 0         │ 'YES'         │
│ 3       │ 'case-4' │ 'Ambiguous Request'     │ 'Make store experience significantly ...' │ 'needs_clarification'    │
'60%'      │ 0       │ 0          │ 1         │ 'YES'         │
│ 4       │ 'case-5' │ 'Insufficient Evidence' │ 'Sync orders to internal enterprise E...' │ 'uncertain'              │
'45%'      │ 0       │ 1          │ 6         │ 'YES'         │
└─────────┴──────────┴─────────────────────────┴───────────────────────────────────────────┴──────────────────────────┴────
────────┴─────────┴────────────┴───────────┴───────────────┘
All 5 cases successfully validated against Zod schema and verified deterministic.
```

5. Sample Structured JSON Outputs

A. Partial Overlap Output (Case 3)

```
{
  "summary": "The request touches approved deliverables (Product Catalog and Search UI) but introduces substantial new out-of-scope features.",
  "reviewStatus": "partially_out_of_scope",
  "confidenceScore": 0.88,
  "inScopeItems": [{
    "item": "Product Catalog and Search UI",
    "matchedDeliverableId": "del_cat_01",
    "matchedScopeReference": "DEL-CAT-01",
    "reason": "Request references functionality covered under approved deliverable \"Product Catalog and Search UI\"."
  }],
  "outOfScopeItems": [
    { "item": "CSV Bulk Ingestion Engine", "reason": "Not included in approved proposal scope.", "potentialImpact": "Requires streaming CSV parser." },
    { "item": "Automated Supplier Inventory Sync", "reason": "Requires background cron job and external supplier adapter." }
  ],
  "affectedDeliverables": [{ "deliverableId": "del_cat_01", "scopeReference": "DEL-CAT-01", "title": "Product Catalog and Search UI" }],
  "disclaimer": "AI analysis is an advisory draft only. Approval or rejection must be finalized by authorized project participants."
}
```

B. Insufficient Evidence Output (Case 5 - Missing Documentation Flagged)

```
{
  "summary": "Insufficient evidence in the approved proposal to determine feasibility or baseline alignment for requested external integrations.",
  "reviewStatus": "uncertain",
  "confidenceScore": 0.45,
  "inScopeItems": [],
  "uncertainItems": [{
    "item": "On-premise SAP ERP Integration",
    "question": "Can the client provide technical specifications, API schemas, sandbox credentials, and network requirements for the sap integration?",
    "missingEvidence": "The approved proposal scope and deliverables contain zero architecture documentation or interface specifications for \"sap\"."
  }],
  "evidence": [],
  "recommendations": [
    "Request architectural specifications and security protocols from client engineering.",
    "Schedule a technical discovery session before providing any cost or timeline estimates."
  ]
}
```

6. HTTP Mock Routes & Controller Architecture

HTTP endpoints are mounted under `/api/ai/*` in `backend/src/routes.js`:

- `GET /api/ai/cases` — Discovery endpoint listing all 5 canonical cases with metadata and sample requests.
- `GET /api/ai/cases/:caseId` — Fetches full input payload and rationale for a single case.
- `POST /api/ai/cases/:caseId/run` (and `GET`) — Runs mock analysis on a named canonical case.
- `GET /api/ai/cases/run-all` — Batch executes all 5 cases and returns an aggregated health report.
- `POST /api/ai/analyze-mock` — Accepts dynamic payloads, validated with `validate({ body: ChangeAnalysisInputSchema })` and `optionalAuth` middleware.

7. Comprehensive File Inventory & Ownership

File Path	Description & Role in Reviews 1 & 2
`backend/src/contracts/aiAnalysis.contract.js`	Formal Zod schema declarations for input payloads and structured output analysis.
`backend/src/contracts/index.js`	Unified module export index for shared contract schemas.
`backend/src/ai/services/changeAnalysisMock.service.js`	Deterministic rule-based mock engine supporting all 5 case categories with reproducible outputs.
`backend/src/ai/fixtures/changeAnalysisFixtures.js`	Five canonical test cases with complete proposal snapshots, deliverables, and explanations.
`backend/src/ai/scripts/runMockCases.js`	CLI runner executable via `npm run mock:cases` with summary table and determinism verification.
`backend/src/ai/ai.controller.js`	Express controller exposing dynamic mock analysis and test case discovery/execution.
`backend/src/ai/ai.routes.js`	HTTP routes mounted under `/api/ai/*` with Zod request validation and `optionalAuth`.
`backend/src/ai/ARCHITECTURE.md`	Comprehensive architectural specification, boundaries, sequence diagrams, and model mappings.
`backend/src/ai/README.md`	Updated roadmap, test instructions, CLI documentation, and endpoint catalog.
`backend/tests/ai.contracts.test.js`	Unit tests for Zod schema boundary validation and type rejection.
`backend/tests/ai.mockCases.test.js`	Dedicated Jest test suite testing all 5 canonical cases, constraints, and determinism.
`backend/tests/ai.routes.test.js`	Integration tests for `GET /api/ai/cases`, `POST /cases/:id/run`, and `POST /analyze-mock`.

8. Automated Verification & Test Execution Evidence

The entire backend test suite was executed using Jest, confirming 100% test passage across all modules:

```
> scopepilot-backend@1.0.0 test
> jest --detectOpenHandles --runInBand

PASS tests/project.test.js
PASS tests/ai.routes.test.js
PASS tests/validation.test.js
PASS tests/notFound.test.js
PASS tests/health.test.js
PASS tests/ai.mockCases.test.js
PASS tests/ai.contracts.test.js

Test Suites: 7 passed, 7 total
Tests:       48 passed, 48 total
Snapshots:   0 total
Time:        2.192 s
Ran all test suites.
```

9. Git Branching, Remote Status & Push Verification

All changes are organized on the dedicated feature branch **`Vishal`**, pushed to remote `origin/Vishal` on GitHub (`https://github.com/Vivek-teja/ScopePilot.git`). The branch is based directly on top of the latest team backend commit (`3b27aca`) with **zero merge conflicts**, and branch `main` remains untouched for mentor integration:

```
$ git log -n 5 --oneline
92ee490 chore: ignore push helper script in gitignore
ab98d23 docs(review-1-2): add consolidated Review 1 and 2 technical report PDF and HTML
2e6ea18 feat(ai): add optionalAuth middleware to AI mock routes
97f411b feat(ai): Review 2 - rule-based mock engine, 5 canonical test cases, CLI runner, and API routes
3b27aca summary (origin/main)

$ git status
On branch Vishal
Your branch is up to date with 'origin/Vishal'.
nothing to commit, working tree clean
```

10. Mentor Evaluation Questions & Answers (Reviews 1 & 2)

Mentor Question	Prepared Technical Response
Why start with an input/output contract before the model?	Defining Zod schemas first establishes an immutable boundary between frontend consumers and the backend engine. It allows teams to mock and build dashboards in parallel without waiting for LLM or RAG infrastructure.
Why use a rule-based mock before an LLM?	A rule-based mock helps the team confirm the contract, validate edge cases, and establish predictable baseline behavior without adding model latency, costs, or non-deterministic variance.
Can the AI agent approve a change request autonomously?	No. ScopePilot enforces strict human authority. The agent produces an evidence-based draft and risk classification; only the authenticated client and freelancer can make binding decisions via `POST /api/change-requests/:id/decision`.
What is partial overlap?	Part of the request is covered by existing deliverables (e.g. Catalog UI), while other parts introduce substantial new work (e.g. bulk CSV processing or background sync) that require extra pricing and scheduling.
What happens when scope evidence is missing or ambiguous?	The engine refuses to guess or hallucinate. It marks `reviewStatus` as `needs_clarification` / `uncertain`, lists `uncertainItems`, and generates targeted questions for the client.

11. Next Steps: Roadmap to Review 3

1. Introduce text extraction and document chunking for approved proposal PDFs/TXTs under `backend/src/ai/rag/`.
2. Set up local Ollama embeddings wrapper (`nomic-embed-text`, 768 dimensions) under `backend/src/ai/embeddings/`.
3. Configure MongoDB Atlas Vector Search index with metadata filters (`workspaceId`, `projectId`, `status: 'ready'`).
4. Compare natural-language requests against approved-scope passages using vector similarity search, recording both successful semantic matches and retrieval failure cases.